PROJECT REPORT

PARALLEL IMAGE PROCESSING PIPELINE

PROJECT ID: 14 | SECTION: 4H | SPRING 2026

Roll Number	Name	Contribution
24K-0599	**Aqsa Zubair**	Pipeline design, synchronization, GUI
24K-0596	**Areeba Zubair**	Thread pool, buffer management, batch
24K-3011	**Maryam**	Filters, performance comparison, testing

Faculty: Mr. Zulfiqar Ali

1. OBJECTIVE

The primary objective of this project is to design and implement a synchronized, multi-threaded image processing pipeline that demonstrates pipeline parallelism using core Operating System synchronization primitives. The system processes batches of images through four sequential stages — loading, filtering, enhancement, and saving — where each stage operates concurrently on different images, maximizing CPU utilization and throughput.

Specific goals include:

- Implement a multi-stage image processing workflow using POSIX thread pools (pthread).
- Protect shared buffers using mutex locks to prevent race conditions.
- Use semaphores to control pipeline capacity and enforce producer-consumer synchronization.
- Maintain correct image processing order using per-image ID assignment.
- Quantify the performance gain of parallel execution versus sequential processing.
- Provide a real-time graphical interface (Qt5) for live pipeline monitoring.

2. INTRODUCTION

Modern computing demands efficient handling of large volumes of data. Image processing is one such domain where performance is critical — photo editors, surveillance systems, medical imaging, and machine learning pipelines all require fast and reliable image transformations. A naive sequential approach processes one image at a time, leaving CPU cores idle and bottlenecking throughput.

This project addresses that limitation by implementing a Pipeline Parallelism architecture, a well-established concurrency pattern where a workload is divided into stages, and multiple items flow through the pipeline simultaneously — analogous to an assembly line in manufacturing. While image 1 is being enhanced in Stage 3, image 2 is being filtered in Stage 2, and image 3 is being loaded in Stage 1, all at the same time.

The implementation is written entirely in C++ using POSIX threads, OpenCV for image operations, and Qt5 for the graphical interface — all running natively on Linux (Ubuntu). The project directly applies Operating Systems concepts including concurrency, synchronization, thread management, deadlock prevention, and inter-thread communication through shared memory buffers.

3. BACKGROUND

3.1 Pipeline Parallelism

Pipeline parallelism is a form of parallel execution where a task is decomposed into a sequence of sub-tasks (stages), each handled by a dedicated worker. Items move through stages like products on an assembly line. The total throughput is determined by the slowest stage (the bottleneck), but the overall speedup over sequential processing can approach the number of pipeline stages when stages have similar durations.

3.2 PRODUCER-CONSUMER PROBLEM

Each pair of adjacent pipeline stages forms a classic producer-consumer pair. The upstream stage produces data items (image tasks) into a shared buffer, while the downstream stage consumes them. Correct synchronization prevents two critical failure modes: buffer overflow (producer writing to a full buffer) and buffer underflow (consumer reading from an empty buffer). These are solved using two counting semaphores — one for empty slots (decremented before producing) and one for filled slots (decremented before consuming) — combined with a mutex for mutual exclusion on the queue structure itself.

3.3 MUTUAL EXCLUSION AND SEMAPHORES

A mutex (mutual exclusion lock) ensures that only one thread at a time can access the shared queue. Semaphores extend this by allowing threads to block and wake up based on a count, making them ideal for capacity control. The POSIX API provides sem_wait() (decrement, block if zero) and sem_post() (increment, wake a waiting thread), used here for both the empty_slots and filled_slots counters on each SharedBuffer.

3.4 ATOMIC VARIABLES

For lightweight counters that do not require full mutex protection — specifically the per-stage image counts and the pipeline_done flag — C++ std::atomic types are used. These guarantee read-modify-write atomicity at the hardware level without the overhead of mutex lock/unlock, and are safe for one writer / one reader or multiple readers patterns. The GUI reads these counters every 200 ms to update progress bars.

3.5 OPENCV

OpenCV (Open Source Computer Vision Library) provides the image I/O and processing functions used in this project: cv::imread for loading images from disk, cv::imwrite for saving results, cv::cvtColor for color space conversion (BGR to grayscale and back), cv::GaussianBlur for smoothing, cv::Canny for edge detection, and Mat::convertTo for brightness and contrast adjustment.

4. PLATFORM & PROGRAMMING LANGUAGES

4.1 DEVELOPMENT ENVIRONMENT

- Operating System: Linux Ubuntu 22.04 LTS
- Compiler: g++ with C++17 standard (-std=c++17)
- Build System: CMake 3.10+
- IDE / Editor: Visual Studio Code
- Version Control: GitHub

4.2 LANGUAGES AND LIBRARIES

- Language: C++ (only C and C++ permitted per project requirements)
- Threading: POSIX Threads — pthread_create, pthread_join, pthread_mutex_t
- Synchronization: POSIX Semaphores — sem_init, sem_wait, sem_post, sem_destroy
- Image Processing: OpenCV 4.x — imread, imwrite, cvtColor, GaussianBlur, Canny, convertTo, filter2D
- GUI: Qt5 Widgets — QProgressBar, QTimer, QTextEdit, QGroupBox, QLabel
- Filesystem: C++17 std::filesystem for directory iteration
- Timing: std::chrono::high_resolution_clock for benchmark measurements

4.3 CMAKE BUILD CONFIGURATION

CMakeLists.txt automatically detects whether Qt5 is installed and conditionally compiles the GUI module by defining the GUI_ENABLED preprocessor macro. If Qt5 is absent, the project compiles and runs in headless (CLI-only) mode. OpenCV and pthread are always required.

5. METHODOLOGY

5.1 SYSTEM ARCHITECTURE

The pipeline is composed of four processing stages connected by three shared buffers. The data flow is:

Image Folder → [Loader Stage] → Buffer 1 → [Filter Stage] → Buffer 2 → [Enhancer Stage] → Buffer 3 → [Saver Stage] → Output Folder

5.2 PIPELINE STAGES

#	Stage	Threads	Responsibility
Stage 1	**Loader**	LOADER_THREADS	Reads images from disk using cv::imread; sends ImageTask to buffer 1
Stage 2	**Filter**	FILTER_THREADS	Applies grayscale and/or other filters (blur, edge detection, sharpen)
Stage 3	**Enhancer**	ENHANCER_THREADS	Adjusts brightness and contrast using convertTo (alpha/beta adjustment)
Stage 4	**Saver**	SAVER_THREADS	Writes processed images to output folder using cv::imwrite

5.3 SHAREDBUFFER IMPLEMENTATION

The SharedBuffer struct (buffer.h) encapsulates all synchronization primitives for the producer-consumer handoff between two adjacent stages. Each buffer holds:

- std::queue<ImageTask> — the actual image queue
- pthread_mutex_t mutex — guards all queue push/pop operations
- sem_t empty_slots — initialized to BUFFER_SIZE (5); producer waits here if buffer is full
- sem_t filled_slots — initialized to 0; consumer waits here if buffer is empty

The push() operation performs: sem_wait(empty_slots) → mutex_lock → queue.push → mutex_unlock → sem_post(filled_slots). The pop() operation is the mirror: sem_wait(filled_slots) → mutex_lock → queue.front/pop → mutex_unlock →

sem_post(empty_slots). This guarantees deadlock-free operation because semaphores are always acquired before the mutex, and the mutex is held for the minimum critical section only.

5.4 TERMINATION PROTOCOL

The pipeline uses a sentinel / poison-pill strategy for graceful shutdown. The Loader thread sends FILTER_THREADS special ImageTask objects with is_done = true after all real images are enqueued. Each Filter thread, upon receiving a done task, forwards one done task downstream and exits. This cascades through the Enhancer and Saver stages, ensuring all real images are fully processed before any thread terminates.

5.5 FILTER OPERATIONS

The filters module (filters.cpp) implements five independent image transformations, each operating on a cv::Mat and returning a new processed Mat:

- apply_grayscale — converts BGR to gray and back to 3-channel BGR for pipeline compatibility
- apply_blur — applies Gaussian blur with a configurable kernel size (must be odd)
- apply_edge_detection — runs Canny edge detector (thresholds 80/150) on a grayscale copy
- apply_brightness — uses Mat::convertTo with alpha (contrast) and beta (brightness offset)
- apply_sharpen — convolves with a 3×3 sharpening kernel using cv::filter2D

In the main pipeline, the Filter stage applies apply_grayscale and the Enhancer stage applies apply_brightness(1.2, 10). Additional filter combinations are available for testing via apply_all_filters().

5.6 GUI DESIGN

When Qt5 is available, the pipeline launches PipelineWindow (gui.cpp) on the main thread while a dedicated pthread runs the processing pipeline in the background. The GUI contains four colored progress bars (one per stage), a live elapsed-time label, a status indicator, and a scrollable log panel. A QTimer fires every 200 ms, calling update_display() which reads the four std::atomic counters and refreshes all UI elements. When pipeline_done becomes true, the timer stops and the status changes to 'COMPLETE'.

5.7 BENCHMARK MODE

Running the binary with the --benchmark flag triggers sequential processing first (run_sequential in performance.cpp), then the parallel pipeline. Both times are measured using

std::chrono::high_resolution_clock and the speedup ratio is computed as sequential_time / parallel_time. The results are printed in a formatted comparison table to stdout.

5.8 OS CONCEPTS APPLIED

OS Concept	Implementation in This Project
Process & Thread Management	POSIX pthread library used to create and manage thread pools for each of the four pipeline stages
Concurrency	Multiple images are processed simultaneously at different pipeline stages, enabling true parallel execution
Synchronization	Mutex locks (pthread_mutex_t) protect shared buffers, ensuring only one thread modifies the queue at a time
Semaphore Mechanism	Two semaphores per buffer (empty_slots & filled_slots) implement the classic producer-consumer pattern
Deadlock Prevention	Semaphores are acquired in a consistent order; mutex is held for the shortest critical section only
OS-Level Scheduling	Thread priority and scheduling are handled by the Linux kernel; threads block on semaphores when buffers are full/empty
Inter-Thread Communication	Shared buffers (SharedBuffer struct with queues) act as the communication channel between stages

5.9 DATA STRUCTURES

Data Structure	Purpose	Synchronization
SharedBuffer (queue)	Holds ImageTask items between pipeline stages	pthread_mutex_t + sem_t
pthread_mutex_t	Guards read/write access to buffer queues	Used with lock/unlock calls
sem_t (empty_slots)	Counts remaining empty slots in buffer; producer waits on this	sem_wait / sem_post

Data Structure	Purpose	Synchronization
sem_t (filled_slots)	Counts waiting images in buffer; consumer waits on this	sem_wait / sem_post
ImageTask (struct)	Holds image ID, filename, cv::Mat, and done flag	Atomic counter updates
std::atomic<int>	Thread-safe counters per stage without mutex overhead	images_loaded/filtered/enhanced/saved
std::atomic<bool>	Signals GUI and main thread that pipeline completed	pipeline_done flag

5.10 ADDITIONAL FEATURES

Feature	Description	Status	File
Multiple Filter Options	apply_grayscale, apply_blur, apply_edge_detection, apply_brightness, apply_sharpen	**Implemented**	filters.cpp
Batch Processing	Automatically collects all .jpg/.png/.bmp files from input folder using std::filesystem	**Implemented**	main.cpp
Performance Comparison	run_sequential() runs a single-threaded pass; print_comparison() outputs a side-by-side table	**Implemented**	performance.cpp
Error Handling	Empty image check in every filter; invalid file detection in loader and sequential mode	**Implemented**	filters.cpp / pipeline.cpp
Progress Visualization (GUI)	Qt5 window with 4 live progress bars + log panel, refreshing every 200 ms via QTimer	**Implemented**	gui.cpp

6. RESULTS

6.1 BENCHMARK RESULTS

Performance benchmarks were conducted on Ubuntu 22.04 with an Intel Core i5 (4 logical cores). Images were standard JPEG photographs at approximately 3–5 MB each. Each test was run three times and the average is reported.

Images Processed	Sequential Time (s)	Parallel Time (s)	Speedup Factor
5	3.21	1.47	**2.18×**
10	6.58	2.43	**2.71×**
20	13.04	4.12	**3.16×**
50	31.90	9.15	**3.49×**

The results demonstrate consistent speedup that increases with batch size. With small batches (5 images), thread creation and synchronization overhead is proportionally larger, yielding a 2.18× speedup. With larger batches (50 images), the pipeline saturates and the overhead amortizes, achieving approximately 3.49× speedup — close to the theoretical maximum of 4× for a four-stage pipeline.

6.2 SPEEDUP ANALYSIS

Speedup follows Amdahl's Law: the portion of work that cannot be parallelized (file I/O, OS scheduling latency, buffer synchronization) limits the maximum achievable speedup. In this implementation the I/O-bound Loader and Saver stages create a mild bottleneck. Increasing LOADER_THREADS and SAVER_THREADS to 2 each showed a further 8–12% improvement in I/O-heavy workloads but introduced the need for careful done-signal propagation (one done signal per consumer thread), as noted in the commented-out configuration in pipeline.h.

6.3 SYNCHRONIZATION CORRECTNESS

All test cases passed without race conditions, deadlocks, or buffer overflows:

- No image was processed out of order (verified by ID logging).
- No image was dropped or duplicated (images_loaded == images_saved for all valid inputs).

- Corrupted / empty images were safely skipped with a warning message and did not stall the pipeline.
- The pipeline terminated cleanly in all runs; no threads were left hanging.

6.4 GUI PERFORMANCE

The Qt5 GUI updated smoothly at 5 Hz (200 ms intervals) without blocking the pipeline. The atomic read operations in update_display() introduced negligible overhead. The log box auto-scrolled correctly and the elapsed timer remained accurate throughout pipeline execution.

7. CONCLUSION

This project successfully designed and implemented a fully functional Parallel Image Processing Pipeline demonstrating core Operating Systems concepts through a practical, real-world application. The four-stage pipeline — Loader, Filter, Enhancer, Saver — processes images concurrently using POSIX threads, with shared buffers protected by mutexes and semaphores implementing the classic producer-consumer pattern.

Key achievements of this project:

- Correct multi-threaded pipeline with zero race conditions, deadlocks, or data corruption across all test scenarios.
- A speedup of 2.18× to 3.49× over sequential processing, scaling with batch size as expected by pipeline parallelism theory.
- A complete filter library supporting grayscale conversion, Gaussian blur, Canny edge detection, brightness/contrast adjustment, and image sharpening.
- Robust error handling that gracefully skips corrupted or missing images without crashing.
- A real-time Qt5 GUI providing live progress visualization for all four pipeline stages simultaneously.
- A benchmark mode enabling direct quantitative comparison between sequential and parallel execution.

The project reinforced that synchronization correctness is the most critical and challenging aspect of multi-threaded programming. Even small mistakes in semaphore initialization or mutex ordering can cause subtle bugs that only manifest under high load. Careful design — acquiring semaphores before the mutex, using the poison-pill termination pattern, and keeping critical sections minimal — proved essential for a correct and efficient implementation.

Future enhancements could include dynamic thread pool sizing based on CPU core count, support for video frame processing, a more sophisticated filter selection UI, and GPU-accelerated image processing using OpenCV's CUDA backend.

8. REFERENCES

- Silberschatz, A., Galvin, P. B., & Gagne, G. Operating System Concepts, 10th Edition. Wiley.
- Linux man pages: pthread_create(3), pthread_mutex_init(3), sem_init(3), sem_wait(3).
- OpenCV Documentation — https://docs.opencv.org/4.x/
- Qt5 Documentation — https://doc.qt.io/qt-5/
- GNU C++ Standard Library Reference — https://cppreference.com
- CMake Documentation — https://cmake.org/documentation/
- Patterson, D. A., & Hennessy, J. L. Computer Organization and Design: The Hardware/Software Interface. Morgan Kaufmann.